

面向对象程序设计

期末串讲

周奕航 2023.12.23



1 导论

编程语言分类：脚本语言、面向机器语言、面向过程语言、**面向对象语言**

程序运行方式：编译执行、解释执行、**编译和解释相结合（如 Java）**

JVM 提供的 JIT 编译器（Just-in-time Compiler）

面向对象思想

万事万物皆对象，

程序由类组成，

对象是类的实例、是细粒度的，

对象之间通过消息相互作用。

Object-Oriented

“物件导向”

2 类和对象(1/3)

对象：属性 / 成员变量 / 字段 + 方法 / (成员) 函数

↓
抽象

类：类是对象的模板 / 类是对象的缔造者 / 类是对象的抽象 / 类是对象的类型

```
struct A {  
    int a;  
};
```

```
int main() {  
    struct A a;  
    struct A *p = (struct A*)malloc(sizeof(struct A));  
    return 0;  
}
```

```
public class A {  
    private int a;  
    public static void main(String[] args) {  
        A a = new A();  
    }  
}
```

2 类和对象(2/3)

如何理解“消息”

- 对象提供的服务是由对象的方法来实现的，因此发送消息实际上也就是调用一个对象的方法。
- 例如：遥控器向电视机发送“开机”消息，意味着遥控器对象调用电视机对象的开机方法
 - **television.open();** //遥控器对象调用电视机对象的开机方法

```
struct TV {  
    int id;  
};
```

```
void open_tv(struct TV *p) {  
    printf("打开了 TV %d\n", p->id);  
}
```

```
int main() {  
    struct TV my_tv = { 1 };  
    open_tv(&my_tv);  
    return 0;  
}
```

```
public class TV {  
    private int id;  
    public TV(int _id) { id = _id; }  
    public void open() { System.out.println("TV " + id + " 打开了"); }  
  
    public static void main(String[] args) {  
        TV my_tv = new TV(1);  
        my_tv.open();  
    }  
}
```

2 类和对象(3/3)

类之间的关系

关联：双向关系，分为一对一、一对多、多对多。（较为普遍）

依赖：一个类依靠另一个类的服务。（但并不要求有包含关系，举例：电脑和路由器）

聚合：弱关联。（举例：公司和职员）

组合：强关联。（举例：汽车和轮胎）

泛化 / 继承：派生、扩充、变异。（举例：动物和狗）

实现：涉及 Java 独有的“接口”概念（举例：Flyable 和鸟、飞机）

3 Java 语言

Java Runtime Environment (JRE , Java 运行环境)

= JVM + Java SE 标准类库

Java Development Kit (JDK , Java 开发工具包)

= JRE + 开发工具集 (如 javac 编译工具等)

3 Java 语言：关键字和保留字

51 个关键字

基本数据类型、类型的大致范围、越界问题、强转问题

boolean , char , byte , float , int , long , short , double , void , enum

程序控制流

do , while , for , if , else , break , continue , return , switch , case , default

类及接口

class , interface , private , protected , public , implements , extends

修饰词

static , abstract , final , transient , synchronized , volatile , strictfp , native

try , catch , finally , throw , throws , assert , instanceof **异常、断言、类型判断**

文件组织

import , package

new , super , this

3 个直接量 true , false , null

2 个保留字 goto , const

3 Java 语言：引用数据类型

要点一：从指针到引用，理解“指向”关系

```
struct A {  
    int i;  
    double d;  
};  
  
int main() {  
    struct A *p = (struct A*)malloc(sizeof(struct A));  
    p->i = 1;  
    p->d = 3.14;  
    return 0;  
}
```

```
public class A {  
    int i;  
    double d;  
    public static void main(String[] args) {  
        A a = new A();  
        a.i = 1;  
        a.d = 3.14;  
    }  
}
```

```
struct A {  
    int i;  
    double d;  
};  
  
int main() {  
    A *p = new A();  
    p->i = 1;  
    p->d = 3.14;  
    return 0;  
}
```


3 Java 语言：引用数据类型

要点二：理解“属性是对象的，方法是类的”

```
struct A {  
    int i;  
};  
  
void f(struct A *p) {  
    p->i = 10;  
}  
  
int main() {  
    struct A a;  
    f(&a);  
    return 0;  
}
```

```
struct A {  
    int i;  
    void f() {  
        this->i = 10;  
    }  
};  
  
int main() {  
    A a;  
    a.f();  
    return 0;  
}
```

```
public class A {  
    int i;  
    void f() {  
        this.i = 10;  
    }  
  
    public static void main(String[] args) {  
        A a = new A();  
        a.f();  
    }  
}
```

3 Java 语言：引用数据类型

要点三：理解引用作为管理者的身份

深拷贝？浅拷贝？

```
public class A {  
    int i = 1;  
    public static void main(String[] args) {  
        A a = new A();  
        A b = a;  
        b.i = 2;  
        System.out.println(a.i);  
  
        String str1 = "Hello";  
        String str2 = "Hello";  
        System.out.println(str1 == str2);  
    }  
}
```

3 Java 语言：函数重载 (Overload)

同名、参数不同、返回值无所谓、修饰符无所谓 (函数签名不同)

```
public class A {  
    void func(char c) { System.out.println("A::func(char c)"); }  
    void func(double d) { System.out.println("A::func(double d)"); }  
    public static void main(String[] args) {  
        A a = new A();  
        a.func(10);  
    }  
}
```

```
public class B {  
    void func(char c) { System.out.println("B::func(char c)"); }  
    void func(int i) { System.out.println("B::func(int i)"); }  
    public static void main(String[] args) {  
        B b = new B();  
        b.func(3.14);  
    }  
}
```

```
public class C {  
    void func() { }  
    int func() { return 1; }  
    public static void main(String[] args) {  
        C c = new C();  
        c.func();  
        int a = c.func();  
    }  
}
```

3 Java 语言：构造（Constructor）与垃圾回收

**特点：与类同名，无返回值（也不是 void）、多数情况要重载
默认构造和无参构造？有参构造？**

new 的作用：分配空间；调用构造；返回引用

**垃圾自动回收机制（Garbage Collection）：Java 虚拟机后台线程负责
System.gc() 和 Runtime.gc()**

判断存储单元是否为垃圾的依据

匿名对象：无管理者（无栈内存引用指向它）

使用场景：只需要进行一次方法调用 / 作为参数传递给函数

3 Java 语言：static

静态成员变量：类加载时就被装载和分配，不依赖于对象而存在 => 通过 类名.变量名 访问

静态成员方法：可以通过 类名.函数名 调用，只能访问类的静态成员

静态代码块：仅能定义在类中，一般用于初始化类的静态变量

静态成员方法、静态代码块中的变量？

语句执行顺序

第一次 new 时：（1）初始化有显式初始化的静态成员变量；（2）顺序执行静态代码块；（3）初始化有显式初始化的非静态成员变量；（4）顺序执行非静态代码块；（5）调用构造。

之后再 new 时：（1）初始化有显式初始化的非静态成员变量；（2）顺序执行非静态代码块；（3）调用构造。

4 封装 (Encapsulation)

修饰符	同一个类	同一个包	子类	整体
private	可见	不可见	不可见	不可见
default / 没有	可见	可见	不可见	不可见
protected	可见	可见	可见	不可见
public	可见	可见	可见	可见

protected

(1) 包内可见。

(2) 对子类可见 (子类和父类在同一个包 : 通过自己访问、通过父类访问。在不同包 : 仅可通过自己访问。)

(3) protected 的 static 成员对所有子类可见。

5 继承 (Inheritance)

构造方法不能继承。

子类构造方法必须最先调用父类构造。

new 出子类对象时，构造方法按照继承链从上往下依次调用。

变量隐藏：子类隐藏父类的同名变量。

总结：super 和 this 的使用

```
super.age = 10;
```

```
super.show();
```

```
super("Tom", Jobs.MANAGER);
```

```
this.age = 100;
```

```
this.func();
```

```
this("Hello", 20);
```

5 继承：方法重写 / 覆盖 (Override)

条件

- (1) 子类中方法的返回值必须与父类的相同、或是其子类。
- (2) 方法名、形参列表完全相同。
- (3) 访问权限不能更低，只能相同或更高。
- (4) 抛出异常的范围不能更大。

注意事项

- (1) `private` 方法不能继承，所以不能覆盖，只有隐藏。
- (2) 构造方法不能继承，所以不能覆盖。
- (3) 静态方法不存在覆盖，只有隐藏。
- (4) `final` 修饰的是最终方法，不能覆盖。

5 多态 (Polymorphism)

引用类型：编译时和运行时。

静多态：发生在编译时，一般指方法重载和隐藏。

动多态：发生在运行时，一般指动态调用方法。

动多态的三个条件：继承、覆盖、Upcasting（父类引用管理子类对象）

5 多态：这是多态吗？

```
class A {  
    public static void func() { System.out.println("A::func()"); }  
}  
  
class B extends A {  
    public static void func() { System.out.println("B::func()"); }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        B b = new B();  
        A a = b;  
        a.func();  
        b.func();  
    }  
}
```

5 多态：instanceof

更多情形

- (1) 方法返回子类引用，用父类引用管理；
- (2) 容器模板为父类型，元素实例是子类型。

Upcasting 总是安全的，有优点也有缺点。

Downcasting 是有风险的，可能产生运行时错误。

A instanceof B，返回一个 boolean 类型的判断结果

A：引用类型

B：类名 / 接口名

instanceof 为 true 的情形

A 管理 B 类的对象

B 为 A 的直接或间接父类

B 为 A 实现的接口

5 多态：强转可能导致错误

强转可能发生编译错误，也可能在运行时抛出 `ClassCastException`

前者，`instanceof` 也不能通过编译；后者，`instanceof` 返回 `false`

// 毫不相关的类之间强制转型

```
Random random = new Random();
```

```
String s = (String)random;
```

// 用接口管理未实现该接口的 `final` 类

```
Integer integer = new Integer(10);
```

```
List list = (List)integer;
```

// 用一个 `final` 类管理其未实现的接口

```
List list = new ArrayList<Integer>();
```

```
String s = (String)list;
```

// 用子类管理父类

```
Vector<Integer> vector = new Vector<Integer>();
```

```
Stack<Integer> stack = (Stack<Integer>)vector;
```

// 用接口管理未实现该接口的普通类

```
Random random = new Random();
```

```
List list = (List)random;
```

// 用未实现某个接口的普通类管理该接口

```
List list = new ArrayList<String>();
```

```
Random random = (Random)list;
```

// 用一个接口管理另一个接口

```
List list = new ArrayList<String>();
```

```
Runnable runnable = (Runnable)list;
```

6 抽象 (Abstract)

关键词 `abstract` 的语义：尚未实现。

含有抽象方法的类，必须用 `abstract` 修饰。抽象类不能实例化。

抽象方法

- (1) 无函数体 (区别函数体为空)
- (2) 必须在抽象类中
- (3) 必须在子类中实现，除非子类也是抽象的

抽象方法不能被 `private final static` 修饰。

7 接口 (Interface)

接口 interface 的语义：方法特征集合，类的行为规范。

接口中所有方法，默认、也必须是 public abstract 的。

接口中所有变量，默认、也必须是 public static final 的。

接口没有构造方法。

native 关键字？

接口回调：用接口类型的引用管理实现了该接口的类。

接口可以继承：同一个函数只能实现一次；不同接口的同名变量相互隐藏；接口变量和类中成员同名时，存在作用域问题。

7 集合框架

8 异常

9 文件 IO

Let's see PDF.

Any questions?